SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool *(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 3 **Total Marks:50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I **__keerthana.G(192521446)__** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation**, including **lexical, syntax, semantic, logical, and runtime errors**.

Given Program

```
#include <stdio.h>

int sumArray(int arr[], int n) {
   int i, sum = 0;
   for(i = 0; i <= n; i++) {
      sum = sum + arr[i];
   }
   return sum;
}

int main() {
   int arr[5] = {1,2,3,4,5}
   int total;

   total = sumArray(arr);

   if(total == 15) {
      printf("Correct Sum\n")
   }

   int a = 10, b = 0;
   int c = a / b;

   int *p;
   *p = 20;
```

```
    return 0;
}
```

Tasks:

(a) Error Identification (Analysis Level)

The given program contains errors at different stages of compilation. The errors are identified below based on their location, nature, and effect on program execution.

Line No.	Error Identified	Nature of Error	Description
51	i <= n	Logical Error	The loop accesses arr[n], which is outside the valid array range when i == n. The condition should be i < n.
58	Missing semicolon after {1,2,3,4,5}	Syntax Error	The array declaration is not terminated with a semicolon.
61	sumArray(arr)	Semantic Error	The function requires two arguments, arr and n, but only one argument is supplied
64	Missing semicolon after printf()	Syntax Error	The printf() statement is not terminated with a semicolon.
68	a / b where b = 0	Runtime Error	Division by zero occurs during execution and produces undefined/exceptional behavior.
71	*p = 20	Runtime Error	Pointer p is declared but not initialized to a valid memory address before dereferencing.

Note: There is no direct lexical error in the given program because the keywords, identifiers, operators, and symbols used are valid lexical tokens.

(b) Classification of Errors (Understanding + Analysis)

The identified errors can be classified according to the compiler phase in which they are detected or their effect on the program.

1. **Lexical Errors**

   No significant lexical error is present in the given program.
   Lexical analysis converts the source program into tokens such as keywords, identifiers, operators, constants, and punctuation symbols. The tokens in this program are lexically valid.

2. Syntax Errors

The following are syntax errors:

- **Line 58:** Missing semicolon after array initialization.
- **Line 64:** Missing semicolon after the printf() statement.

Syntax errors violate the grammatical structure of the C language.

3.Semantic Errors

- **Line 61:** sumArray(arr) is called with only one argument, whereas the function definition requires two arguments: arr and n.

This is a semantic error because the statement does not satisfy the required meaning and function interface.

4. Logical Errors

- **Line 51:** The condition i <= n is logically incorrect for an array containing n elements.

The valid indexes of an array containing n elements are from 0 to n-1. Therefore, the correct condition is:
i < n

5. Runtime Errors

- **Line 68:** Division by zero.
- **Line 71:** Dereferencing an uninitialized pointer.

These errors may only become apparent when the corresponding statements are executed.

(c) Compilation Behavior (Evaluation Level)

1. Errors Detected During Compilation

The following errors can be detected during compilation:

Line 58 – Missing semicolon

int arr[5] = {1,2,3,4,5}

The compiler expects a semicolon after the declaration.

Correct form:

int arr[5] = {1,2,3,4,5};

Line 61 – Incorrect number of function arguments

total = sumArray(arr);

The function is defined as:

int sumArray(int arr[], int n)

Therefore, two arguments must be supplied.

Correct form:

total = sumArray(arr, 5);

Line 64 – Missing semicolon

printf("Correct Sum\n")

Correct form:

printf("Correct Sum\n");

2. Errors That Appear During Execution

Division by Zero

```
int a = 10, b = 0;
int c = a / b;
```

Since b is zero, the division operation is invalid during execution.

Invalid Pointer Dereference

```
int *p;
*p = 20;
```

The pointer p has not been initialized to a valid memory location. Dereferencing it may cause a segmentation fault or other undefined behavior.

Array Boundary Violation

The loop uses:

```
for(i = 0; i <= n; i++)
```

When i == n, the program attempts to access:

```
arr[n]
```

which is outside the valid range of the array.

3. Effect of Early-Stage Errors

Compiler processing generally occurs through several stages. Lexical analysis produces tokens, syntax analysis checks grammatical structure, semantic analysis checks meaning, and later stages generate code.

If a syntax error such as a missing semicolon is encountered, the compiler may be unable to correctly construct the program structure. Therefore, later semantic analysis or code generation may be affected.

As a result, errors such as division by zero or invalid pointer access may not be reached during execution because the program may fail to compile first.

(d) Code Correction (Application Level)

The corrected program is given below

```
#include <stdio.h>

int sumArray(int arr[], int n) {
   int i, sum = 0;

   for(i = 0; i < n; i++) {
      sum = sum + arr[i];
   }

   return sum;
}

int main() {
   int arr[5] = {1,2,3,4,5};
   int total;

   total = sumArray(arr, 5);

   if(total == 15) {
      printf("Correct Sum\n");
   }

   int a = 10, b = 2;
   int c = a / b;

   int value = 20;
   int *p = &value;

   *p = 20;
```

```
    printf("Result of division: %d\n", c);

    return 0;
}
```

Corrections Made

1. Changed i <= n to i < n.
2. Added the missing semicolon after the array declaration.
3. Passed both required arguments to sumArray().
4. Added the missing semicolon after printf().
5. Changed b = 0 to b = 2 to prevent division by zero.
6. Initialized pointer p with the address of a valid variable.
7. Maintained proper syntax and semantics throughout the program.

(e) Impact Analysis (Evaluation Level)

1. Improper Pointer Usage

The statement:

```
int *p;
*p = 20;
```

declares a pointer without assigning it a valid address. When the pointer is dereferenced, the program attempts to write to an unknown memory location.

This can result in:

- Memory corruption
- Program crashes
- Undefined behavior

A safe implementation is:

```
int value = 20;
int *p = &value;
*p = 20;
```

2. Division by Zero

The original program contains:

int a = 10, b = 0;

int c = a / b;

Division by zero is mathematically undefined and causes an error during program execution.

Its possible effects include:

- Program termination
- Runtime exception
- Undefined behavior
- Incorrect execution

A safe program should check the denominator before performing division.

3. Array Boundary Violation

The original loop is:

for(i = 0; i <= n; i++)

For an array containing five elements, valid indexes are:

0, 1, 2, 3, 4

When i becomes 5, the program attempts to access arr[5], which is outside the allocated array.

This may cause:

- Incorrect results
- Access to unrelated memory
- Memory corruption
- Undefined behavior

The correct loop condition is:

for(i = 0; i < n; i++)

(f) Advanced Compiler Perspective (Create Level)

A compiler can be improved by providing better diagnostic facilities and performing additional analysis before program execution.

1. Better Error Messages

The compiler should provide detailed and user-friendly error messages containing:

- Line number
- Column number
- Error type
- Cause of the error
- Suggested correction

For example:

Error at line 61:

Too few arguments to function 'sumArray'.

Expected: 2 arguments.

Provided: 1 argument.

Suggestion: use sumArray(arr, 5).

Such messages help programmers identify and correct errors quickly.

2. Early Detection of Runtime Errors

Some potential runtime errors can be identified before execution using static analysis. The compiler can detect or warn about:

- Possible division by zero
- Uninitialized variables
- Uninitialized pointers
- Array bounds violations
- Unreachable code
- Infinite loops in detectable cases

This reduces the possibility of runtime failures.

3. Enhancement 1 – Static Analysis

A static-analysis module can inspect the source code without executing it. It can identify possible unsafe operations and generate warnings before code generation.

For example:

int b = 0;

int c = a / b;

can generate a warning such as:

Warning: Possible division by zero.

4. Enhancement 2 – Intelligent Debugging Support

The compiler can integrate advanced debugging information such as:

- Exact source location
- Variable values
- Function call information
- Memory-access information
- Suggested fixes
- Error dependency information

This allows programmers to understand not only **where** an error occurred but also **why** it occurred.

Corrected Code :

```
#include <stdio.h>
int sumArray(int arr[], int n) {
   int i, sum = 0;

   for (i = 0; i < n; i++) {
      sum = sum + arr[i];
   }

   return sum;
}

int main() {
   int arr[5] = {1, 2, 3, 4, 5};
   int total;
```

```
    total = sumArray(arr, 5);

    if (total == 15) {
        printf("Correct Sum\n");
    }

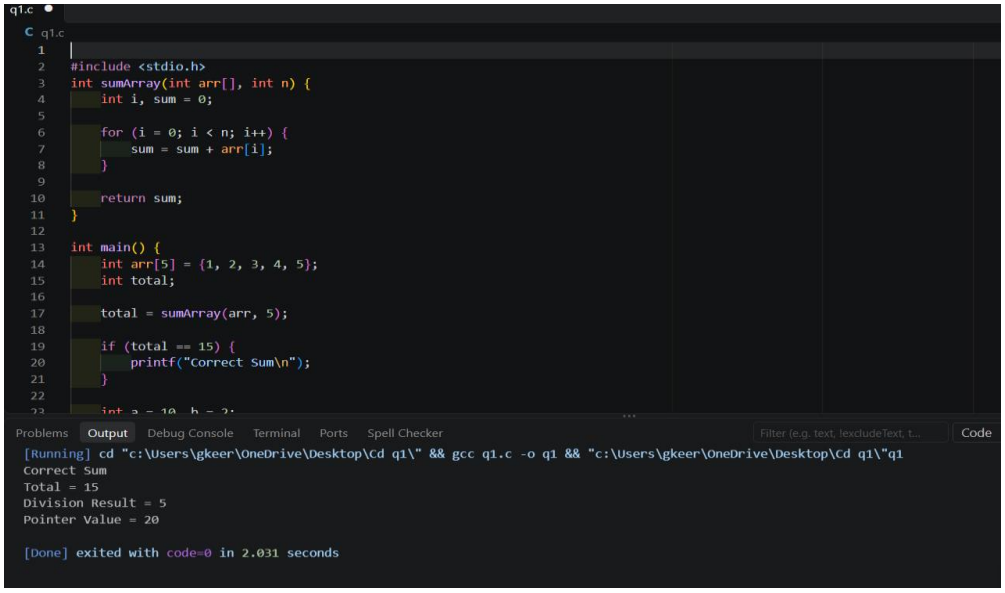
    int a = 10, b = 2;
    int c = a / b;

    int x = 20;
    int *p = &x;
    *p = 20;

    printf("Total = %d\n", total);
    printf("Division Result = %d\n", c);
    printf("Pointer Value = %d\n", *p);

    return 0;
}
```

Output Screenshot :



Result:

Thus, the errors in the given C program were corrected, and the program was successfully executed in VS Code with the expected output.

Q2. The following C program contains **multiple lexical errors** introduced intentionally. These errors occur due to **invalid tokens, incorrect identifiers, malformed constants, and illegal symbols**, which are typically detected during the **lexical analysis phase** of a compiler.Carefully examine the program and answer the questions given below.

```
#include <stdio.h>

int main() {
    int 1num = 100;
    float rate@ = 25.5;
    char ch = 'abc';
    int total = 50$20;
    int num#1 = 30;
    int %count = 40;
    printf("Welcome to Compiler Design);
    int x = 09;
    int y = 0xZ12;
    int z = 12.3.4;

    char str1[] = "Hello;
    char str2[] = "World";

    int _valid = 10;
    int invalid-id = 20;

   printf("Value: %d\n", total);
```

Tasks

(a) Error Identification

Line	Invalid Tokens	Error
3	1num	Identifier starts with a digit
4	rate@	@ is illegal in a C identifier
5	‘abc’	Character constant contains multiple characters
6	50$20	$ is not a standard C operator/symbo
7	num#1	# cannot occur inside an identifier
8	%count	% cannot start an identifier
9	"Welcome to Compiler Design)	Unterminated string literal
10	09	Invalid octal integer constant
11	0xZ12	invalid hexadecimal constant
12	12.3.4	Malformed floating constant
13	"Hello	Unterminated string literal
16	invalid-id	- is interpreted as minus, not part of identifier
17	5..6	Malformed floating constant
18	10@5	Illegal @ symbol
19	'xy'	Character constant contains multiple characters
20	3value	Identifier starts with a digit

(b) Error Classification

Line	Invalid Tokens	Rule violated	Error
3	1num	Identifier formation	Identifier starts with a digit
4	rate@	Identifier formation	@ is illegal in a C identifier
5	‘abc’	Character literal formation	Character constant contains multiple characters
6	50$20	Operator formation	$ is not a standard C operator/symbo
7	num#1	Identifier formation	# cannot occur inside an identifier
8	%count	Identifier formation	% cannot start an identifier
9	"Welcome to Compiler Design)	String literal formation	Unterminated string literal
10	09	Integer constant formation	Invalid octal integer constant
11	0xZ12	Hexadecimal constant formation	invalid hexadecimal constant
12	12.3.4	Floating-point constant formation	Malformed floating constant
13	"Hello	String literal formation	Unterminated string literal
16	invalid-id	Identifier formation	- is interpreted as minus, not part of identifier
17	5..6	Floating-point constant formation	Malformed floating constant
18	10@5	Operator formation	Illegal @ symbol
19	'xy'	Character literal formation	Character constant contains multiple characters
20	3value	Identifier formation	Identifier starts with a digit

(C) errors occured

Invalid Token	Corrected Token
1num	num1
rate@	rate
'abc'	‘a’
50$20	50+20
num#1	num1
%count	count
"Welcome to Compiler Design)	"Welcome to Compiler Design”
09	9
0xZ12	0xA12
12.3.4	12.34
"Hello	“Hello”
invalid-id	Invalid_id
5..6	5.6
10@5	10+5
'xy'	‘x’
3value	Value3

(d) Compiler analysis

A lexical analyzer, also called a scanner, is the first phase of a compiler. It reads the source program character by character and groups them into tokens such as keywords, identifiers, constants, operators, and symbols.

When the scanner encounters an invalid token, it detects the violation of lexical rules and reports an error with the corresponding line number and invalid token.

Example:

int 1num = 100;

The scanner identifies 1num as an invalid identifier because identifiers cannot begin with a digit. It reports a lexical error and may attempt to continue scanning the remaining program.

(ii), (iii) and (iv) Does the compiler stop immediately or continue scanning using error recovery?

A compiler does not always stop immediately when it encounters a lexical error. Its behavior depends on the compiler implementation and the severity of the error.

1. Stops immediately

- Some compilers or compilation tools terminate when a serious error is detected.
- Further compilation may be prevented if the source code cannot be processed correctly.

2. Continues scanning using error recovery

- Many compilers report the error and continue scanning the remaining source code.
- The scanner may skip invalid characters or tokens and resume at a recognizable point.
- This helps identify multiple errors in one compilation instead of correcting them one at a time.

Example:

int 1num = 100;

int rate = 25;

The compiler may report the invalid identifier 1num and continue scanning the second declaration.

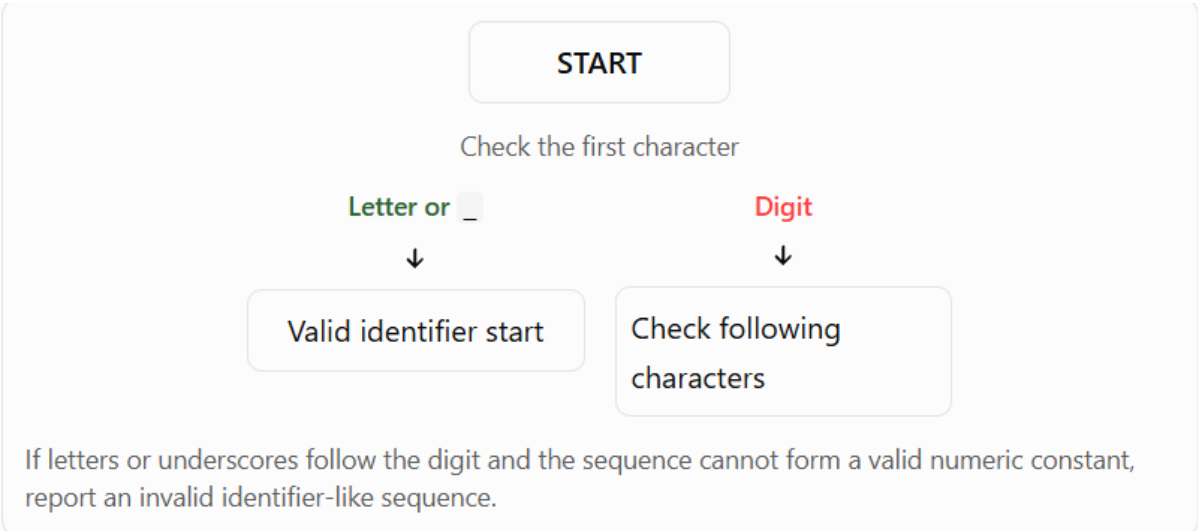
Conclusion: Most modern compilers attempt error recovery whenever possible, allowing programmers to identify and correct multiple errors in a single compilation. However, some errors, such as an unterminated string literal, can disrupt scanning and cause additional error messages.

(e) Advanced Thinking

(i) Design a simple rule or DFA idea to detect invalid identifiers starting with digits

Rule: In C, an identifier must begin with an alphabetic letter (A–Z, a–z) or an underscore (_). It cannot begin with a digit (0–9).

A simple DFA (Deterministic Finite Automaton) can detect invalid identifier-like sequences using the following logic:



Examples:

- 1num → Invalid identifier
- 3value → Invalid identifier
- num1 → Valid identifier
- _value → Valid identifier

This rule helps the lexical analyzer detect identifier naming errors before the syntax analysis phase.

(ii) Suggest how error messages can be improved for beginners

Compiler error messages should be simple, clear, and helpful. They should include the line number, invalid token, reason for the error, and a suggested correction.

Example 1: Invalid identifier

Instead of:
Error: Invalid token

Display:
Line 3: Invalid identifier '1num'.
Identifiers cannot begin with a digit.
Suggestion: Use 'num1'.

Example 2: Invalid numeric constant

Instead of:
Error: Invalid number

Display:
Line 10: Invalid integer constant '09'.
Octal numbers cannot contain the digit 9.
Suggestion: Use '9' for a decimal integer.

Example 3: Missing quotation mark

Line 13: Unterminated string literal.
A closing double quotation mark is missing.
Suggestion: Write "Hello".
Corrected code:

```c
#include <stdio.h>

int main() {
    int num1 = 100;
    float rate = 25.5;
    char ch = 'a';
    int total = 5020;
    int num1value = 30;
    int count = 40;

    printf("Welcome to Compiler Design\n");

    int x = 9;
    int y = 0x12;
    float z = 12.34;

    char str1[] = "Hello";
    char str2[] = "World";

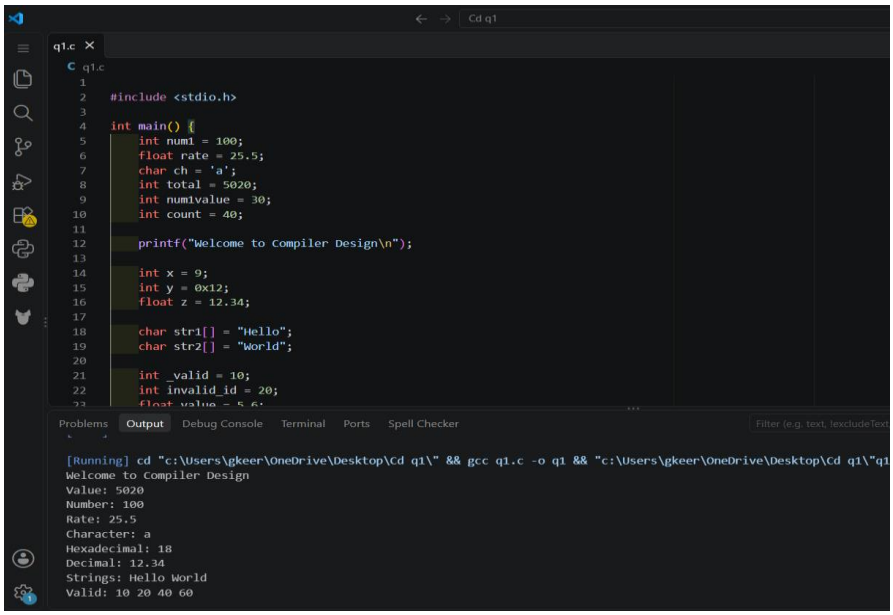
    int _valid = 10;
    int invalid_id = 20;
    float value = 5.6;
    int data = 105;
    char c = 'x';
    int value3 = 60;

    printf("Value: %d\n", total);
    printf("Number: %d\n", num1);
    printf("Rate: %.1f\n", rate);
    printf("Character: %c\n", ch);
    printf("Hexadecimal: %d\n", y);
    printf("Decimal: %.2f\n", z);
    printf("Strings: %s %s\n", str1, str2);
```

```
    printf("Valid: %d %d %d %d\n", _valid, invalid_id, count, value3);

    return 0;
}
```

Output:



Result:

Thus, the lexical errors in the given C program were identified and corrected, and the corrected program was successfully compiled and executed in VS Code with the expected output.

Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains **multiple errors distributed across different phases of a compiler**, including:

A. Lexical Errors

B. Syntax Errors

C. Semantic Errors

D. Intermediate Code Generation Issues (logic/precedence)

E. Code Optimization Issues (redundancy/inefficiency)

F. Runtime / Target Code Errors

Carefully analyze the program and answer the questions below.

```
#include <stdio.h>

int divide(int a, int b) {
   return a / b
}

int main() {
   int #value = 20;
   float num = 15.5
   int result;

   result = divide(10);

   if(result = 5) {
      printf("Result is 5\n")
   }

   int x = 10;
   int y = 0;
   int z = x / y;

   int arr[4] = {1,2,3,4};
   printf("%d\n", arr[4]);

   int *ptr;
   *ptr = 25;

   int total;
   total = x + y * result + 0;

   while(x < 20) {
```

```
        printf("%d\n", x);
    }

    return 0;
}
```

Do the following Tasks

(a) Identify Errors (Analysis Level)

Error identification table :

Line	Statement / Error	Description
4	return a / b	Missing semicolon after the return statement.
8	int #value = 20;	Invalid symbol # used in an identifier.
9	float num = 15.5	Missing semicolon after the declaration.
11	result = divide(10);	Function requires two arguments, but only one is supplied.
12	if(result = 5)	Assignment operator is used instead of the comparison operator ==.
13	printf("Result is 5\n")	Missing semicolon after the printf statement.
17	int z = x / y;	Division by zero because y is initialized to 0.
19	printf("%d\n", arr[4]);	Array index is out of bounds; valid indices are 0 to 3.
21	*ptr = 25;	Pointer is uninitialized before dereferencing.
23	total = x + y * result + 0;	Redundant addition of zero; an optimization opportunity
24	while(x < 20)	x is never incremented, causing an infinite loop.

(b) Classify by Compiler Phase

1. Lexical errors

Lexical errors occur when characters do not form valid tokens according to C language rules.

Line	Error	Explanation
8	#value	The # symbol cannot be used as part of an identifier.

Correct statement:

int value = 20;

2. Syntax errors

Syntax errors occur when statements violate the grammatical rules of C.

Line	Error	Explanation
4	Missing ;	The return statement must end with a semicolon.
9	Missing ;	The variable declaration must end with a semicolon.
13	Missing ;	The printf statement must end with a semicolon.

3. Semantic errors

Semantic errors occur when code violates language constraints or does not express the intended operation correctly.

Line	Error	Explanation
11	divide(10)	The function requires two arguments, but only one is provided.
12	result = 5	Assignment is used instead of comparison, probably unintentionally.

Correct statements:

result = divide(10, 2);

```
if (result == 5) {
   printf("Result is 5\n");
}
```

4. Intermediate code generation issues

Intermediate code generation converts source expressions into a simpler internal representation.

Line 23:
total = x + y * result + 0;
C operator precedence evaluates multiplication before addition. Therefore, the expression means:
Total = x + (y * result) + 0;
There is no actual precedence error in the given expression. However, if the programmer intended addition before multiplication, parentheses would be required:
total = (x + y) * result;

5. Code optimization issues

Line 23 contains a redundant operation:
total = x + y * result + 0;
Adding zero does not change the value of the expression. The compiler can simplify it to:
total = x + y * result;
This is called algebraic simplification.

6. Runtime errors

Runtime errors occur during program execution or involve unsafe behavior that may manifest at runtime.

Line	Error	Explanation

17	x / y	Division by zero; behavior is undefined in C.
19	arr[4]	Accesses an element outside the array bounds.
21	*ptr = 25	Dereferences an uninitialized pointer, causing undefined behavior.
24–26	Infinite loop	x remains unchanged, so the loop condition stays true.

(c) Compilation vs Execution (Evaluation)

1. Errors detected at compile time

Error	Line	Detection
Invalid #value	8	Lexical or syntax diagnostic
Missing semicolon	4, 9, 13	Syntax analysis
Incorrect function argument count	11	Semantic/constraint checking
Assignment in condition	12	May produce a warning; valid syntax
Potential division by zero	17	May be detected by compiler warnings or static analysis
Potential out-of-bounds access	19	May be detected by static analysis
Uninitialized pointer	21	May be detected by compiler warnings or static analysis

The missing semicolons and invalid identifier can prevent successful compilation. The function argument mismatch also requires correction for a valid program.

2. Errors that appear during execution

- Division by zero at line 17.
- Out-of-bounds array access at line 19.
- Invalid pointer dereference at line 21.
- Infinite loop at lines 24–26.

These issues are not guaranteed to produce a clear runtime error message. In C, the first three involve undefined behavior, while the infinite loop can cause the program to run indefinitely.

3. Why do some errors prevent further compilation?

The compiler must understand the source code's tokens, syntax, and types before generating executable code.
For example, a missing semicolon can confuse syntax analysis, while an incorrect function call violates a function's declared requirements. The compiler may attempt error recovery to report additional problems, but it generally cannot produce a valid executable until the required errors are corrected.

(d) Correct the Program (Application)

The following program fixes the syntax and semantic errors, removes the redundant operation, and avoids the unsafe runtime behavior.

```
#include <stdio.h>

int divide(int a, int b) {
   return a / b;
}

int main() {
   int value = 20;
   float num = 15.5;
   int result;

   result = divide(10, 2);

   if (result == 5) {
      printf("Result is 5\n");
```

```
    }

    int x = 10;
    int y = 2;
    int z = x / y;

    int arr[4] = {1, 2, 3, 4};
    printf("%d\n", arr[3]);

    int ptr_value = 0;
    int *ptr = &ptr_value;
    *ptr = 25;

    int total;
    total = x + y * result;

    while (x < 20) {
        printf("%d\n", x);
        x++;
    }

    printf("z = %d\n", z);
    printf("total = %d\n", total);
    printf("Pointer value = %d\n", *ptr);

    return 0;
}
```

Corrections made

1. Added missing semicolons.
2. Replaced #value with value.
3. Supplied both arguments to divide().
4. Changed = to == in the condition.

5. Changed y from zero to 2.
6. Changed arr[4] to arr[3].
7. Initialized the pointer with a valid address.
8. Removed the redundant + 0.
9. Added x++ to terminate the loop.

(e) Deep Analysis (Evaluation)

1. How incorrect pointer usage affects runtime

Original code:

```
int *ptr;
*ptr = 25;
```

The pointer is declared but not initialized. It does not point to a known valid object. Dereferencing it attempts to write to an indeterminate memory location, which results in undefined behavior and may cause a crash or memory corruption.

Safe version:

```
int value = 0;
int *ptr = &value;
*ptr = 25;
```

Here, the pointer refers to a valid integer variable.

2. How array bounds violations affect memory

Original code:

```
int arr[4] = {1, 2, 3, 4};
printf("%d\n", arr[4]);
```

The valid indices are 0, 1, 2, and 3. Accessing arr[4] goes beyond the array's valid range.

Possible consequences include:

- Reading an unrelated memory value.
- Producing unpredictable output.
- Causing a crash.

- Triggering other undefined behavior.

Correct code:

printf("%d\n", arr[3]);

3. How operator precedence affects intermediate code

Consider:

total = x + y * result;

Multiplication has higher precedence than addition. The compiler therefore evaluates y * result first.

A simplified intermediate representation is:

t1 = y * result

t2 = x + t1

total = t2

If the intended calculation were to add x and y first, the programmer would write:

total = (x + y) * result;

Parentheses change the expression structure and therefore the intermediate representation generated by the compiler.

(f) Advanced Task (Create Level)

1. Two improvements in compiler design

Improvement 1: Static analysis and runtime checking

Compilers and development tools can detect potential problems earlier using warnings, static analysis, and sanitizers.

Examples:

- Detect possible division by zero.
- Identify uninitialized pointers.
- Detect array bounds violations where possible.
- Warn about suspicious infinite loops.

Improvement 2: Beginner-friendly error messages

Error messages should display the line number, the reason for the error, and a suggested correction.

For example:

Line 11: Function 'divide' requires 2 arguments,

but only 1 argument was provided.

Suggested correction:

result = divide(10, 2);

This makes errors easier to understand and correct.

2. How the optimization phase removes redundant expressions

The optimization phase simplifies expressions that do not affect the result.

Original expression:

total = x + y * result + 0;

Intermediate representation:

t1 = y * result

t2 = x + t1

t3 = t2 + 0

total = t3

After optimization:

t1 = y * result

total = x + t1

The optimizer uses the algebraic identity:

$$A + 0 = A$$

This removes unnecessary operations, reduces generated instructions, and can improve execution efficiency.

Corrected code:

```
#include <stdio.h>

int divide(int a, int b) {
   return a / b;
}

int main() {
   int value = 20;
   float num = 15.5;
   int result;

   result = divide(10, 2);

   if (result == 5) {
      printf("Result is 5\n");
   }

   int x = 10;
   int y = 0;
   int z = 0;

   if (y != 0) {
      z = x / y;
   } else {
      printf("Division by zero is not allowed\n");
   }

   int arr[4] = {1, 2, 3, 4};
   printf("%d\n", arr[3]);

   int data = 25;
   int *ptr = &data;
   *ptr = 25;

   int total;
   total = x + y * result;
```

```
    while (x < 20) {
        printf("%d\n", x);
        x++;
    }

    printf("Value = %d\n", value);
    printf("Number = %.1f\n", num);
    printf("Division Result = %d\n", result);
    printf("Total = %d\n", total);
    printf("Pointer Value = %d\n", *ptr);

    return 0;
}
```

Output:

```c

#include <stdio.h>

int divide(int a, int b) {
    return a / b;
}

int main() {
    int value = 20;
    float num = 15.5;
    int result;

    result = divide(10, 2);

    if (result == 5) {
        printf("Result is 5\n");
    }

    int x = 10;
    int y = 0;
    int z = 0;

```

```
Result is 5
Division by zero is not allowed
4
10
11
12
13
14
15
16
17
18
19
Value = 20
Number = 15.5
Division Result = 5
Total = 10
Pointer Value = 25
```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand